Лабораторная работа №1

Краткая теория и объяснение кода

1. Строки и символы в C++

`std::string` — стандартный класс для хранения и работы со строками. Строку можно последовательно перебрать и получить каждый её символ типа `char`.

В этой работе гласными считаются только строчные символы `a`, `e`, `i`, `o`, `u`. Пробелы и остальные буквы пропускаются.

Для чтения строки используется:

```
std::getline(std::cin, text);
```

`std::getline` читает все символы до конца строки, включая пробелы. Обычный ввод через `std::cin >> text` для задания не подходит, поскольку он остановился бы на первом пробеле.

2. Объявление и реализация функции

Объявление функции сообщает компилятору её имя, тип возвращаемого значения и параметры. Оно находится в заголовочном файле `include/vowels.hpp`.

Реализация содержит сам алгоритм и находится в `src/vowels.cpp`. Такое разделение позволяет использовать одну и ту же функцию и в основной программе, и в тестах, не дублируя её код.

Функция имеет следующий вид:

```
std::size_t countVowels(const std::string& text);
```

- `std::size_t` — беззнаковый целочисленный тип, подходящий для количества элементов;
- `const std::string&` — ссылка на неизменяемую строку: функция не копирует строку и не изменяет её.

3. Алгоритм подсчёта

Алгоритм последовательно проверяет каждый символ:

1. Начальное количество гласных равно нулю.
2. Для каждого символа строки выполняется проверка через `switch`.
3. Если символ является гласной, счётчик увеличивается на единицу.
4. После просмотра всей строки функция возвращает счётчик.

> Сложность алгоритма — **O(n)**, где `n` — длина строки: каждый символ проверяется один раз. Дополнительная память — **O(1)**, поскольку независимо от длины строки хранится

> только счётчик и текущий символ.

4. Пояснения к файлам проекта

include/vowels.hpp

- `#pragma once` не допускает повторного включения заголовочного файла при компиляции.
- `<cstddef>` предоставляет тип `std::size_t`.
- `<string>` необходим для использования `std::string` в объявлении функции.
- Здесь находится только объявление `countVowels`, без реализации.

src/vowels.cpp

```
for (const char ch : text) {
    switch (ch) {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
            ++count;
            break;
    }
}
```

Это цикл по всем символам строки. Несколько `case` подряд используют один общий блок кода: при совпадении с любой гласной выполняется `++count`.

`break` завершает только `switch`, после чего цикл переходит к следующему символу. Если символ не является гласной, счётчик не изменяется.

main.cpp

Основная программа выполняет три действия:

1. Создаёт переменную `std::string text`.
2. Читает всю введённую строку через `std::getline`.
3. Вызывает `countVowels(text)` и выводит возвращённое число.

В `main.cpp` нет алгоритма подсчёта: основная программа использует готовую функцию из библиотеки `vowels`.

tests/vowels_test.cpp

Google Test используется для автоматической проверки результата. Макрос `TEST` создаёт отдельный тест, а `EXPECT_EQ` сравнивает фактический результат функции с ожидаемым.

Проверяются пять случаев:

- пустая строка — `0`;
- строка без гласных — `0`;
- строка `aeiouaeiou` — `10`;
- строка с пробелами `this is a test` — `4`;
- обычный текст `hello world` — `3`.

CMakeLists.txt

CMake выполняет следующие задачи:

- проверяет, что выбран компилятор GCC 14;
- включает стандарт C++17;
- собирает `src/vowels.cpp` как библиотеку `vowels`;
- собирает программу `vowels_app` из `main.cpp` и подключает библиотеку;
- загружает Google Test через `FetchContent`;
- собирает `vowels_tests` и регистрирует тесты в CTest.

Команда `cmake --build build` компилирует все цели, а `ctest --test-dir build --output-on-failure` запускает зарегистрированные тесты и показывает их результаты.